This supplement answers a concrete question the main text raises but settles elsewhere: when belief sharing is routed through an actual transport channel instead of a direct function call, does the fused consensus change? It specifies that transport — the same single-host interface sec. 8.21 names as the anchor for eventual multi-machine federation — and establishes that, under a lossless round-trip, the answer is no.

Concretely, the transport realizes belief sharing over a real in-memory channel rather than a direct aggregation call. Each worker holds a local posterior q_n over the shared latent factor and serializes it to lossless IEEE-754 float64 bytes using the numpy-lossless-float64 encoding (numpy's native array format), guaranteeing bit-identical round-trip across the transport boundary. A server collects 5 such beliefs, fuses them with the same robust server step at robustness $c = 1.5$, and broadcasts the consensus q back to every contributing worker over its response channel.

Proposition (Federation bit-identity)

When the transport serialization is lossless — an exact IEEE-754 float64 round-trip — the federated consensus equals the in-process aggregation $q = \mathrm{robust\_aggregate}(\{q_n\}, c)$ **bit-for-bit*. Transport moves bytes, not mathematics, so no precision is lost and no result changes.*

Because the round-trip is exact, this implementation retires the direct in-process serialization caveat. The queue adapter remains a genuine `queue.Queue` transport, `run_multiprocess_round` exercises the same server/worker protocol with one OS worker process per agent on a single machine, and `run_socket_round` exercises the loopback-TCP adapter. The fused result is provably unchanged. Bit-identity verified: True.

The implementation lives in the `federation/` package. The end-to-end and socket transport tests exercise the full round-trip — worker serialization, server aggregation, consensus broadcast, out-of-order arrival, the single-machine process helper, loopback TCP framing, optional HMAC frame integrity, and file-backed digest-verified replay validation — and assert bit-identity against the in-process `robust_aggregate` result. A caller-owned SQLite guard rejects reused round IDs across local process restarts, but does not define a shared multi-host replay domain. This test surface is the API contract that sec. 8.21 identifies as the anchor for future network transport: the aggregation mathematics can remain unchanged, but true multi-machine work still requires cross-host transport, identity-bound mTLS, shared replay state, discovery, restart orchestration, and threat-model validation that this single-host evidence does not supply.